

Artifact Appendix of the Pape Entitled: Random Robust Secret Sharing with Perfect Privacy and its Applications

Mohammad Hassan Ameri Jeremiah Blocki
Purdue University

ACM CCS 2026

1 Artifact Appendix

This appendix summarizes the purpose, availability, contents, dependencies, and execution of the artifact accompanying our paper. It describes the hardware, software, and configuration requirements, identifies the main experimental claims supported by the artifact, and provides instructions for evaluating them. By linking these claims to concrete functionality checks and performance experiments, the appendix helps evaluators and readers reproduce the reported results and reuse the implementation.

1.1 Abstract

In this artifact, we describe how to install, build, and evaluate our Random Robust Secret Sharing (RRSS) implementation and its two applications: conditional encryption for arbitrary Hamming distance and fuzzy password-authenticated key exchange using RRSS (RRSS-fPAKE). The evaluation workflow covers three main components: building the project and exploring toy examples for RRSS and conditional-encryption functionality; evaluating the conditional-encryption construction; and benchmarking RRSS-fPAKE against the prior RSS-fPAKE implementation.

We provide instructions for installing the dependencies, compiling the C++ implementation and its Python binding, and running the basic functionality demonstrations. We also explain how to evaluate performance by reproducing the conditional-encryption results in Figure 2 and the fPAKE measurements in Table 1, subject to the reduced benchmark coverage described below.

The accompanying `README.md`, included in the artifact archive publicly available on Zenodo (<https://doi.org/10.5281/zenodo.22802638>), provides detailed setup instructions, commands, and expected outputs. Further access information is provided in the *How to access* section. To simplify evaluation, the artifact includes shell scripts that invoke the compiled C++ test programs and Python benchmarks, collect measurements, calculate summary statistics, and generate tables and plots. The `README` () also explains how to run individual experiments, select the supported security levels, modify the toy examples, and regenerate tables or figures from existing measurements.

1.2 Description & Requirements

1.2.1 Artifact contents and organization

The artifact contains C++ RRSS and conditional encryption implementations for arbitrary Hamming distance, an RRSS Python binding (to be considered as a python library as well), RSS-fPAKE and

RRSS-fPAKE benchmarks, and workflows for Table 1 and Figure 2. The plots and table in this appendix contain original paper measurements, not newly executed benchmarks as a reference for the respected artifact reviewers. The archive contains the complete RRSS source tree. Evaluators should begin with `README.md` (provided in the artifact directory of the project), which gives the authoritative step-by-step instructions. The principal components are:

- `RandomRobustSS.cpp` and related C++ sources: the RRSS implementation, including sharing, reconstruction, and valid-share identification;
- `build.sh` and `CMakeLists.txt`: reproducible project configuration and build entry points;
- `external/fPAKE-benchmark/`: implementations of RRSS-PAKE (as well as RSS-fPAKE for the sake of comparison with prior work of fastZip), configuration files, prepared inputs, and statistics programs;
- `testCondEncArbHam/`: Catch2 functionality and evaluation cases for conditional encryption;
- `artifact/scripts/`: dependency installation and automated reproduction scripts; and
- `artifact/results/reproduced/`: outputs created by the reproduction workflows.

1.2.2 Security, privacy, and ethical concerns

Benchmarks run locally, with fPAKE parties communicating over localhost. Installation and data acquisition need Internet access. The paper uses public sanitized datasets and reports aggregate measurements; do not substitute personal passwords. Dependency installation requires administrative privileges and modifies system packages, so a disposable Ubuntu environment is recommended. The Figure 2 workflow removes earlier raw measurement files; archive any results that must be retained. This artifact is a research prototype, not a production authentication system.

1.3 More Discussion on Ethical concerns

Our experimental evaluation uses only publicly available, sanitized datasets released by prior work to support reproducible research. In particular, when comparing the empirical performance of our conditional encryption scheme with [AB24], we use password typo datasets from [CAAJR16], which are publicly available at <https://pages.cs.wisc.edu/~chatterjee/projects/pwtypos.html>. These datasets were generated in the user studies of [CAAJR16], where participants were asked to type randomly sampled passwords from the RockYou breach. Because these datasets are partially derived from breached-password data, we briefly discuss the ethical considerations of our use of them here.

The running time of our conditional decryption algorithm is independent of the underlying password and typo distribution, once password length is fixed. Thus, the performance of our conditional encryption scheme could in principle be evaluated using synthetic typo data. However, the conditional decryption algorithm of [AB24] depends significantly on the password typo distribution making it valuable to evaluate performance on a realistic password typo dataset. Accordingly, to enable a fair comparison and to maximize reproducibility, we use the same dataset as [AB24] in our evaluation.

To minimize risk, we do not directly use any breached dataset beyond the breached passwords that may already appear in the publicly released datasets of [CAAJR16]. Moreover, we report only aggregate statistics from our performance analysis and do not publish any individual passwords.

We therefore do not believe that our use of these datasets causes additional harm to users. At the same time, the fPAKE and conditional encryption protocols we develop are intended to improve user security and privacy.

We also compare the empirical performance of our fPAKE protocol with the implementation of the weaker fPAKE protocol from [FHALHH21]. For this comparison, we use the same datasets as [FHALHH21]. Fomichev et al. [FHALHH21] collected these datasets from sensor measurements (accelerometer, gyroscope, and barometer) taken inside four different cars across a range of scenarios, including city roads, country roads, highways, and parking garages. These datasets were released publicly to support reproducibility. We do not believe that using these publicly available datasets raises significant ethical concerns, as they do not appear to contain private or confidential information.

1.3.1 How to access

The permanent archival version of our artifact provided for the ACM-CCS26 is available on Zenodo at <https://doi.org/10.5281/zenodo.22801889>. The Zenodo archive contains the complete evaluator-facing source tree, documentation, reproduction scripts, and the prepared RRSS-fPAKE benchmark inputs required for the documented workflows.

The actively maintained development repository is available at the Github repository with link <https://github.com/mhassanameri/RRSS>. Evaluators should use the Zenodo archive for artifact evaluation and begin with `artifact/README.md`, which provides the authoritative installation, functionality check, and reproduction instructions. When obtaining the source from GitHub rather than Zenodo, run `download_rrsfPAKE_data.sh`, as documented in the artifact README, to obtain the large prepared benchmark-input archive.

1.3.2 Hardware dependencies

No specialized cryptographic accelerator is required. Both protocol parties were executed locally to eliminate network variability and isolate protocol-induced overhead. We evaluated the performance our implementation of fPAKE using RRSS on a Ubuntu 24.04.3 LTS (Noble Numbat) with a 3.10 GHz 15-Core Intel® Core™ i9-9900 CPU processor and 64 GB 2666 MHz DDR4 RAM memory.

1.3.3 Software dependencies

The artifact was tested on Ubuntu 24.04. It requires a C++20-compatible compiler, CMake, Python 3 with development headers, GMP, NTL, Crypto++, Protocol Buffers, pybind11, Catch2, NumPy, Cryptography, PyCryptodome, and Matplotlib. The script `artifact/scripts/install_dependencies_ubuntu24.sh` installs the required system and Python dependencies on a fresh Ubuntu 24.04 environment. No proprietary software is needed.

1.3.4 Benchmarks

The Table 1 fPAKE comparison uses the same prepared FastZIP-derived fingerprint inputs as the original RSS-fPAKE implementation: `acc_h-bar` and `acc_h-gyrW`, with fingerprint length $n = 36$, maximum Hamming distance $\ell = 3$, and security parameters $\lambda \in \{128, 244\}$. Both parties communicate over localhost.

The Figure 2 conditional-encryption evaluation uses the public sanitized password-typo dataset released by Chatterjee et al. The artifact evaluates input lengths $n \in \{8, 16, 32, 64, 128\}$ and applicable Hamming-distance thresholds. It compares the RRSS-based arbitrary-Hamming-distance construction with the prior AB24/CCS24 construction.

1.4 Set Up

1.4.1 Installation and build

From the repository root, install the dependencies and build the project:

```
# In a root-owned Ubuntu 24.04 container
bash artifact/scripts/install_dependencies_ubuntu24.sh
```

```
# On a regular Ubuntu system, prefix the previous command with sudo.
bash build.sh
```

The RRSS-fPAKE benchmark imports the compiled Python module `rrss`. Confirm that it is visible to the same Python interpreter used by the benchmark:

```
export PYTHONPATH="$PWD/build${PYTHONPATH:+:$PYTHONPATH}"
python3 -c "import rrss; print(rrss.__file__)"
```

The supplied reproduction scripts perform this check automatically. The README also documents optional `chmod +x` commands for users who prefer invoking scripts directly; all commands in this appendix invoke scripts through `bash` and therefore do not require a permission change.

1.4.2 Functionality checks

After a successful build, run the RRSS smoke test:

```
./build/RRSS
```

It generates shares for a fixed secret, corrupts two shares, reconstructs the secret, and should end with `Match: true`.

For a conditional-encryption functionality demonstration, run:

```
cd build/testCondEncArbHam
./tests "HDArbSemiFuncCheck"
cd ../..
```

The test uses messages at Hamming distance one with threshold $\ell = 4$. The terminal output should include `the recovered payload is: This is good Thing to see!`. Inspect this value: the supplied Catch2 case has no explicit correctness assertions, so its passed status alone does not establish payload recovery. Instructions for modifying the examples are in the README.

1.5 Evaluation Workflow

1.5.1 Major claims

(C1)

On the paper’s measured configurations, RRSS-fPAKE Sender and Receiver execution times remain below one second, with increased communication overhead relative to RSS-fPAKE. Experiment E1 evaluates this performance claim (Table 1). Security is established by the paper’s proofs, not by benchmarks.

(C2)

RRSS enables practical conditional encryption for arbitrary Hamming-distance thresholds. In particular, its conditional-decryption time is essentially independent of ℓ , while the AB24/CCS24 comparison becomes impractical as ℓ grows. This claim is supported by experiment E2 and Figure 2.

1.5.2 Experiments

E1: Reproduce Table 1 (fPAKE performance). **Preparation.** Build the project as above. The archival ZIP includes the prepared inputs. When using GitHub, first run `bash download_rssfPAKE_data.sh`.

Execution. From the repository root, run:

```
bash artifact/scripts/reproduce_table1_all.sh
python3 artifact/scripts/generate_table1.py
```

The first command runs the RSS-fPAKE baseline and RRSS-fPAKE at both security levels. It starts the Receiver locally, runs the Sender, and saves logs and summary statistics. The second command produces `artifact/results/reproduced/table1.csv` and `table1.md`.

Results. Compare calculation time, network time, overall time, and communication overhead with Table 1. Exact timings can vary with hardware and load; the key expected result is the same qualitative comparison: RRSS-fPAKE remains sub-second but has larger communication overhead.

E2: Reproduce Figure 2 (conditional encryption). **Preparation.** Build the project and ensure Matplotlib is installed. The artifact uses reduced repetition counts for evaluation practicality; the paper used larger counts.

Execution. From the repository root, run:

```
bash artifact/scripts/reproduce_figure2.sh
```

The script runs `HDArbSemiEval` and `HDAtmostTSemiEval`, copies raw measurements to `artifact/results/reproduced/`, and invokes `generate_figure2.py`.

Results. The workflow produces raw measurements and normalized CSV files, together with PDF and PNG plots (`figure2.pdf` and `figure2.png`). The panels plot conditional-decryption time by n , conditional-decryption time by ℓ , and regular/conditional ciphertext sizes. Figure 2 below uses the supplied original paper measurements and TikZ sources; compare new measurements with these reference results.

Coverage and runtime. The reduced configuration evaluates our scheme at applicable $\ell \in \{2, 4, 8, 16, 32, 64\}$ with $\ell < n$, but AB24 only at threshold 4 for $n \in \{8, 16, 32, 64\}$ and threshold 3 for $n = 128$. Thus it does not regenerate every AB24 point in the paper figure; full coverage requires additional original-parameter runs. E1 uses three executions per benchmark instance. Setup time, compute time, peak RAM, and disk use for E1/E2 must be measured before final submission.

Table 1: fPAKE performance comparison across security levels, roles, and input types. Password length: $n = 36$. Times are in seconds; original paper measurements.

Algorithm	Sec.	Type	Role	Run time (calc)	Comm. time (net)	Overall time	Role comm. ovhd. (KiB)	Total comm. ovhd. (KiB)
fPAKE RSS	128	acc_h-bar	Sender	0.0123	0.0063	0.0186	3.3008	5.1289
			Receiver	0.0485	0.0119	0.0605	1.8281	5.1289
		acc_h-gyrW	Sender	0.0125	0.0065	0.0190	3.6523	5.6836
			Receiver	0.0626	0.0120	0.0746	2.0313	5.6836
	244	acc_h-bar	Sender	0.0351	0.0141	0.0492	5.0771	8.0303
			Receiver	0.0661	0.0289	0.0951	2.9531	8.0303
		acc_h-gyrW	Sender	0.0363	0.0155	0.0518	5.6162	8.8975
			Receiver	0.0854	0.0300	0.1153	3.2813	8.8975
fPAKE RRSS	128	acc_h-bar	Sender	0.2498	0.0083	0.2580	34.2031	36.0312
			Receiver	0.0447	0.2495	0.2943	1.8281	36.0312
		acc_h-gyrW	Sender	0.3387	0.0112	0.3499	41.7383	43.7695
			Receiver	0.0591	0.3385	0.3975	2.0313	43.7695
	244	acc_h-bar	Sender	0.2682	0.0166	0.2847	35.9795	38.9326
			Receiver	0.0588	0.2614	0.3202	2.9531	38.9326
		acc_h-gyrW	Sender	0.3547	0.0195	0.3742	43.7021	46.9834
			Receiver	0.0740	0.3466	0.4205	3.2813	46.9834

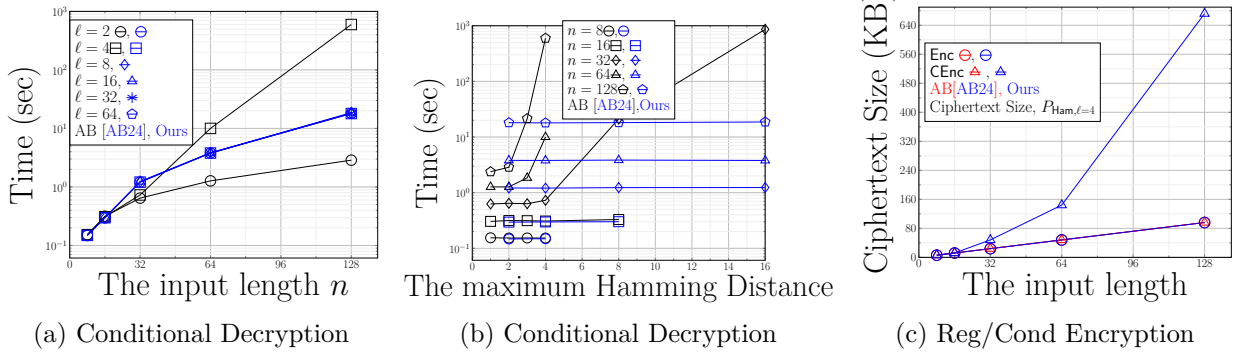


Figure 2: Performance Evaluation of our CE for Arbitrary Hamming Distance. Original paper measurements, rendered from the supplied data and plotting code.

1.6 Notes on Reusability

The RRSS library is modular: its sharing, reconstruction, valid-share identification, finite-field arithmetic, serialization, and Python binding are separated so that researchers can reuse RRSS in additional protocols. The supplied `main.cpp` demonstration can be adjusted to use a different secret or corruption pattern, provided that the configured RRSS length matches the secret length. The conditional-encryption tests can likewise be adapted by editing the inputs in `testCondEncArbHam/testCondEncEvalArbHam.h` and rebuilding the `tests` target.

1.7 Version

This appendix is based on the ACM CCS artifact-appendix organization used for the authors' CCS 2024 artifact, updated for ACM CCS 2026.

References

- [AB24] Mohammad Hassan Ameri and Jeremiah Blocki. Conditional Encryption with Applications to Secure Personalized Password Typo Correction. ACM CCS, 2024.

- [CAAJR16] Chatterjee, Rahul and Athayle, Anish and Akhawe, Devdatta and Juels, Ari and Ristenpart, Thomas. pASSWORD tYPOS and how to correct them securely. IEEE SP, 2016
- [FHALHH21] Mikhail Fomichev, Julia Hesse, Lars Almon, Timm Lippert, Jun Han, and Matthias Hollick. FastZIP: faster and more secure zero-interaction pairing. ACM MobiSys 2021)